

Large Language Models for Behavior Tree Generation in Unreal Engine

Axel Gerbeaud
ISART Digital
Paris, France

Leo Lorcet
ISART Digital
Paris, France

Loan Campan
ISART Digital
Paris, France

Alexander Dockhorn
SDU Metaverse Lab
Odense, Denmark
adoc@sdu.dk

Florian Wolf
ISART Digital
Paris, France
florwolf@gmail.com

Aurélien Lherbier
ISART Digital
Paris, France
a.lherbier@isartdigital.com

Maël Addoum
ISART Digital
Paris, France
m.addoum@isartdigital.com

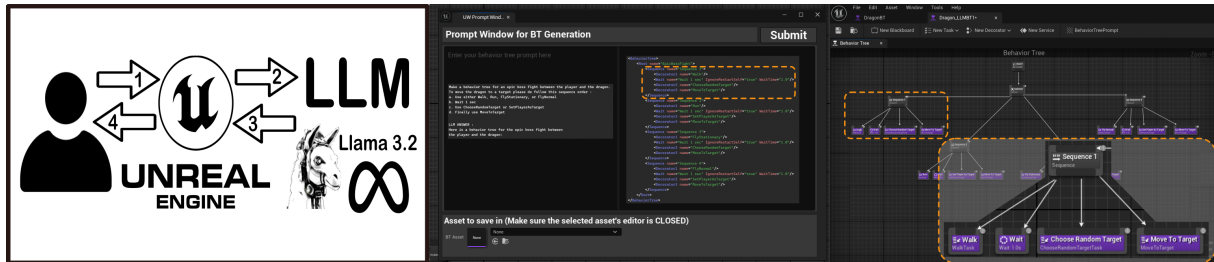


Figure 1: Generation process and result of our LLM-based Behavior Tree Generation in Unreal Engine 5.

Abstract

Behavior Trees (BTs) enable dynamic and strategic gameplay by controlling non-player characters (NPCs) in video games, but their construction and iterative refinement remain time-consuming and technically demanding for game developers. This project explores the integration of Large Language Models (LLMs) within Unreal Engine 5 as a tool to automatically generate BTs from natural language prompts, thereby reducing the complexity of designing game AI and accelerating the production pipeline. In order to choose the adapted model in terms of coherence and practicability for local integration, multiple models were benchmarked. Results revealed that models under 2.5 GB provided the best trade-off between performance and output quality. The LLaMA 3.2 3B Instruct quantized model was found to be particularly suitable. In this study, we have thus developed a tool based on the Llama-Unreal Plugin to implement a local LLM into the engine. A custom prompt interface was implemented in the editor, allowing users to submit natural language inputs. The model responds with an XML description of the BT, which is then parsed and instantiated as a native BT asset

in Unreal Engine. The XML schema was carefully designed to accommodate different node types (tasks, composites, decorators). The implemented system was tested with simple NPC behaviors like seeking and chasing the player for a guardian, or like a boss fight sequence for a dragon. It successfully produced functional BTs from simple prompts in various gameplay scenarios. Early results have shown how LLMs can streamline the creation of Game AI. The system could be extended to support more complex and hierarchical BTs, improve error handling in XML parsing, and integrate with other game engines such as Unity or Godot.

CCS Concepts

- Computing methodologies → Natural language processing;
- Applied computing → Computer games; • Human-centered computing → User interface toolkits.

Keywords

LLM, Behavior Tree, Unreal Engine, prototyping, game industry, production, tool, plugin, collaborative AI-human design

ACM Reference Format:

Axel Gerbeaud, Leo Lorcet, Loan Campan, Alexander Dockhorn, Florian Wolf, Aurélien Lherbier, and Maël Addoum. 2026. Large Language Models for Behavior Tree Generation in Unreal Engine. In *Foundations of Digital Games (FDG '26)*, August 10–13, 2026, Copenhagen, Denmark. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3815598.3815631>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

FDG '26, Copenhagen, Denmark

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2495-4/2026/08

<https://doi.org/10.1145/3815598.3815631>

1 Introduction

Machine Learning (ML) tools have been used in the last years in various contexts within the video game industry. ML-based Procedural Content Generation (ML-PCG) [31] powered by generative AI (GenAI) models is increasingly used in game development to autonomously create a wide range of game assets from consistent gameplay sequences [12, 14] to entire game levels [3, 40]. As games grew in complexity and players became more demanding, the need for more intuitive and powerful PCG tools led to pushing innovations by leveraging Large Language Models (LLMs) into the game production pipeline. Unlike ML-PCG, which focuses on predefined tasks and structured datasets, LLMs can interpret high-level instructions using natural language prompts, thus facilitating a more dynamic and creative development process [8, 34]. In fact, LLMs and transformer models in general have emerged as the central technology in recent years by extending the capabilities of content generation, minimizing the workload, and requiring less human intervention. Many game assets could be generated using natural language as input, such as interactive environments [4, 5, 19], levels [39, 41], music and sound [15, 23], 3D models [18], and non-playable characters (NPCs) [7, 16, 26, 35].

Concerning the development of NPCs, LLM-based approaches have mainly focused on dialogues and contextual conversation with players rather than generating in-game behaviors [8, 30, 32]. Many LLM-based toolkits for Unity and Unreal Engine have been developed to facilitate workflows within the real-time engines. Using ACE technology by Nvidia, Ubisoft showcased LLM-based NPCs in their Neo NPC demo [27], where the player can freely converse with the in-game characters.

When it comes to the behavior of NPCs, less research has been conducted to introduce ML-based techniques. While implementations of artificial intelligence (AI) have been present since the very first days in the video game industry [17], much is based on non-learning-based approaches. For instance, Finite State Machines (FSM) [2, 11, 38] have been used early on in games such as Pac-Man (1980) [21]. Decision-making AI based on symbolic AI is actually quite common in the video game industry for controlling the action plan of Non-Player Characters (NPCs). In this context, Behavior Trees (BTs) are widely used, since their use in the game *Halo 2* [9, 36], to create the logic and the deterministic sequences of NPC actions controlled by decorators [22, 33]. Planning approaches [24] have also been used to control NPC's behaviors, such as Goal Oriented Action Planning (GOAP) [1, 37] and Hierarchical Task Networks (HTNs) [13, 20], however, BTs remain the most used in a vast majority of the released video games. These classical techniques are characterized by rigid, static, and predefined behaviors that involve repetitive and predictable actions, which limit both adaptability and scalability [6].

In contrast, data-driven AI methods based on ML, such as reinforcement learning and genetic algorithms, enable the development of NPCs that can tailor and evolve their strategies based on player

interactions, yielding possibly more immersive and engaging gaming experiences. Examples include the **F.A.C.U.L.**¹ project from Tencent and more recently the **Teammates**² project from Ubisoft.

Here, we propose to combine well-known symbolic AI with GenAI by creating a tool that generates BTs from natural language inputs. Such a hybrid approach has the potential to combine the efficiency of traditional decision-making frameworks with the flexibility and simplicity of using LLMs. In particular, as BTs grow in size and complexity, manually authoring them becomes increasingly tedious and error-prone, making an LLM-based generation approach highly recommendable for accelerating the prototyping of large-scale BTs.

However, it is important to acknowledge that LLMs can hallucinate, potentially producing structurally invalid or semantically incorrect BTs, e.g., referencing non-existing nodes or generating illogical control flows. Such hallucinations may result in non-workable BTs that require manual correction. Therefore, human oversight remains essential when using LLM-generated BTs, especially for safety-critical or complex behaviors.

In this paper, we present our plugin for LLM-based generation of BTs inside of *Unreal Engine* (UE). The tool has been developed for UE 5.5. Developers can describe the intended BT architecture through a prompt window, upon which the module will generate the corresponding tree, allowing for iterative refinement via follow-up prompts. In such a way, when needed, human verification can still be applied during the post-treatment phase to ensure reliability and coherence. This automation system is expected to democratize game development, enabling players and novice developers to control their NPCs' behaviors, providing a more personalized and engaging experience.

In Section 2, the related works are listed. In Section 3, our plugin is presented, giving an overview of its workflow and settings, and of the chosen LLM and its one-shot prompt. Then, in Section 4, the results are presented, and the validity of the generated BT is evaluated within the particular scenario of a boss fight with a dragon. Finally, concluding remarks and perspective are given in Section 5.

2 Related Work

2.1 Robotics

The idea of using LLMs to produce BTs is not entirely new. Izzo *et al.* [10] proposed to use LLMs fine-tuned for generating BTs for robot control. While we use a similar approach for generating BTs for video game NPCs, these characters will have access to a much wider range of actions. Furthermore, some of these actions may be context-sensitive, requiring the BT to include corresponding control structures. For this reason, our approach includes context information with the list of user-defined actions as part of the LLM prompt, resulting in a much more versatile tool. Further, we adopt the XML format by Izzo *et al.* for encoding and decoding BTs. As a result, the solution can be applied independently of the game engine by a corresponding interface.

¹<https://www.tencent.com/en-us/articles/2202068.html>

²<https://news.ubisoft.com/en-us/article/3mWlITuWuu0MoVuR6o8ps/ubisoft-reveals-teammates-an-ai-experiment-to-change-the-game>

2.2 Behavior Tree Generation

Beyond LLM-based approaches, other methods for automatic BT generation have been explored. Evolutionary computation has been applied to evolve BTs for game NPCs. Nicolau *et al.* [25] proposed evolutionary approaches for generating BTs that navigate platform games. Paduraru and Paduraru [28] presented a framework combining BTs with genetic algorithms for automatic difficulty management and testing. More recently, Partlan *et al.* [29] introduced a co-creative evolutionary approach for generating BTs for game NPCs, allowing designers to guide the evolutionary process. While these methods can produce effective BTs, they typically require representing the user’s preferences as a fitness function and multiple evaluation/optimization cycles to generate a matching BT. In contrast, our LLM-based approach generates BTs directly from natural language descriptions and potentially allows for language-based edits, offering a more intuitive interface for rapid prototyping.

2.3 Game Engines

As mentioned earlier, the entertainment industry has leveraged the potential of LLMs by starting to integrate them into the most popular Game Engines and thereby ensuring more scalable and intelligent development workflows that are context-aware and adapted for intricate game worlds. For instance, *Unity* has developed an innovative platform, **Unity AI**³, merging **Unity Sentis**⁴ and **Unity Muse**⁵, to assist content creation (3D objects, animation, sprites, textures, NPC actions, etc.) directly within the Editor by using natural language-based inputs. Very recently, *Unreal Engine* has released **UE 5.7**, which now includes the **Epic Developers Assistant**⁶, which is an AI assistant that integrates directly with the editor. We aim to follow a similar approach to ensure direct integration with developer tools and thereby maximize accessibility.

3 LLM-BT Unreal Engine Plugin

This section describes the LLM-BT plugin we have developed to produce BTs using LLMs. Generally speaking, this approach can be followed with cloud-hosted solutions using (paid) API keys. However, we wanted our tool to be able to run directly on a PC (desktop or laptop) with a consumer-grade graphics card and regular VRAM. Therefore, we developed the prototype with a focus on small-scale (<8B parameters) and locally hosted LLMs⁷. Testing the tool under these conditions helps to evaluate its limits and capabilities. Even if small LLMs, as in their infancy, tend more often to hallucinate, it is a good stress test. Moreover, this ensures a privacy-preserving workflow as no data on the game or the to-be-generated BTs are shared with external sources. This is in line with the preferences of many game studios, who aim to keep data local during the development phase. By focusing on local models, the developer retains full control over the LLM usage and the data being processed, without depending on external API providers whose terms, pricing, or availability may change.

³<https://unity.com/products/ai>

⁴<https://unity.com/products/sentis>

⁵<https://unity.com/products/muse>

⁶<https://dev.epicgames.com/community/assistant/unreal-engine>

⁷Around 25 models were tested which were mainly different quantized version of Mistral 7B, Llama 3.2B instruct and 2.7B chat, Tessa 3B, and DeepSeek distilled Qwen 7B, and last but not least Schematron 3B which derived from Llama 3.2B instruct

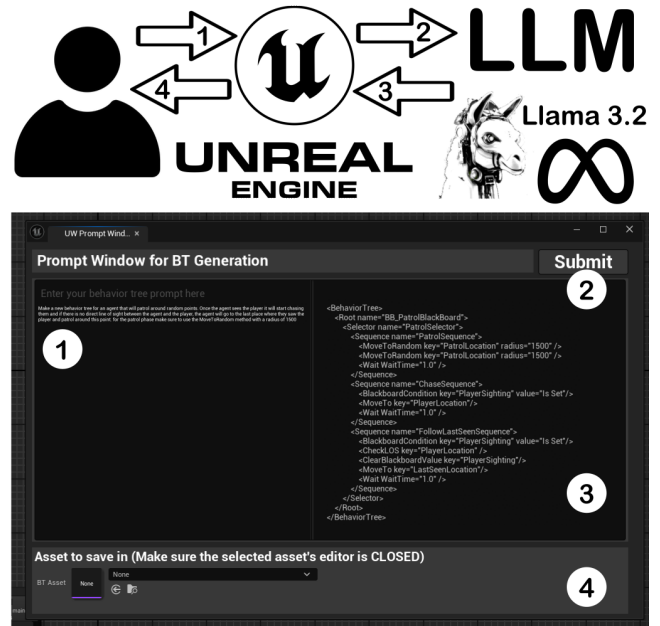


Figure 2: Overview of the four-step plugin workflow: the user provides a natural language description of the desired BT (step 1), which is combined with a context prompt and sent to the local LLM (step 2). The LLM produces an XML-formatted BT (step 3), which is then parsed by the plugin to generate a native BT asset inside UE (step 4). The lower part of the figure shows the corresponding editor interface.

3.1 Plugin workflow

The general workflow of our plugin consists of 4 steps as depicted in Fig. 2. First, the user describes the desired BT using natural language. Then, this user prompt is appended to a context prompt containing the one-shot example of an XML-formatted BT together with a list of basic and user-defined available BT nodes. Corresponding prompt sections are automatically generated using the blueprint and C++ code. Next, the LLM’s response is shown in a window, so that the user can evaluate the resulting BT (in XML format) as well as other comments given by the LLM. Finally, an XML parser is to transform the XML BT into a BT UE asset⁸. Ultimately, the user can keep the prompt window open and reiterate the process to improve or modify the generated BT.

3.2 Llama-Unreal Plugin and Related Modifications

Our plugin is based on the **Llama-Unreal** plugin, which allows us to load local LLMs and interact with them from inside UE. **Llama-Unreal** is essentially a wrapper around the **llama.cpp** library, specifically made to load and use local models from C++ code. As the **Llama-Unreal** plugin is meant to work on a component at

⁸the current prototype requires existing assets to be closed, such that the file can be overridden

```

<BehaviorTree>
  <Root name="ExampleTree">
    <Sequence name="Main Sequence">
      <Sequence name="Move 1">
        <Walk name="Walk" bIgnoreRestartSelf="false"/>
        <Wait name="Pause" bIgnoreRestartSelf="false"/>
        <SetPlayerAsTarget name="Set Target" bIgnoreRestartSelf="false"/>
        <MoveToTarget name="Move To Target" bIgnoreRestartSelf="false"/>
      </Sequence>
    </Sequence>
  </Root>
</BehaviorTree>

```

Figure 3: Base context prompt, providing one-shot example.

run-time, it had to be remade as an editor-time wrapper by adapting the existing code into an independent interface. This also gives more control over Llama itself.

To adapt the plugin, we first created a class in C++ that would interact with the editor. This class contains the function to create the prompt window, as well as calls to the plugin. An editor tick was produced to use the methods linked to Llama. Multi-threaded solutions were used to receive the tokens sent by the model in use. Those tokens then appear in the prompt window as text, making the user able to understand the results and reasoning of the LLM.

3.3 Base context prompt and XML formatted BT

Similarly to the work of Izzo *et al.* [10], an XML format was chosen for the BT generation. One advantage of the XML format is that it is game engine independent; therefore, the present implementation can be transferred to other game engines if required, providing an interface with local LLMs or relying on an API key for cloud-based LLMs. The XML also helps the LLM to structure the BT.

The base context prompt is exposed in the plugin and the user can modify it easily. The beginning of the default base context prompt passed to the LLM prior to the user prompt is:

You generate valid Unreal Engine Behavior Tree XML only.
 Rules:
 - Output only one well-formed XML document.
 - Do not output explanations, comments, markdown, or code fences.

It is then composed of 4 sections:

- **Section 1)** describes the basic structure with an example of how to start and end the XML BT
- **Section 2)** defines the 3 types of tags that can be used, namely **Nodes** (Sequence, Selector, SimpleParallel), **Decorators**, and **Tasks** and gives some rules.
- **Section 3)** gives an example of a valid XML BT using these 3 types of tags (see Fig. 3)
- **Section 4)** consists of project-specific settings and is detailed in the following subsection.

3.4 Plugin settings and Final context prompt

In order to increase usability and versatility, user settings have been introduced. In the project's settings, the user can define which composite nodes, tasks, and decorators the LLM has access to (Fig.

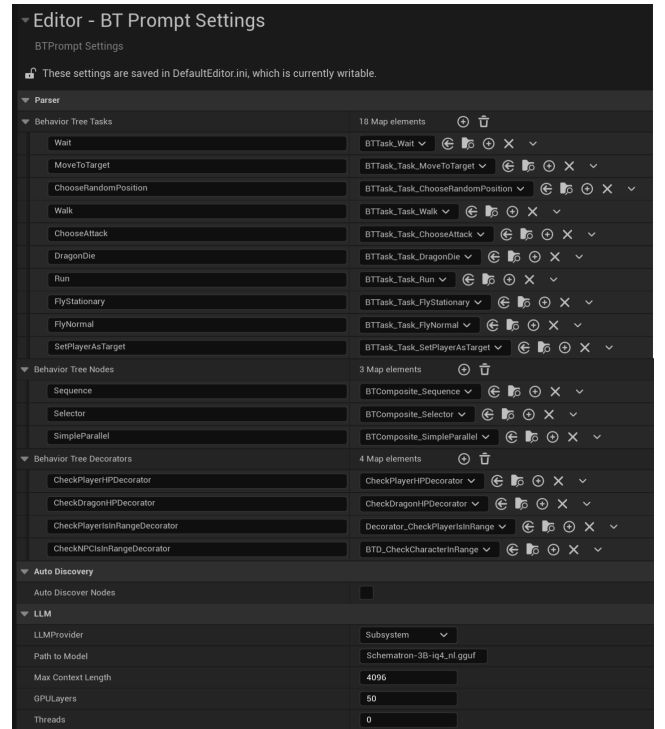


Figure 4: User custom editor's settings allowing to specify to the LLM which user-defined BT nodes, tasks, and decorators are accessible for the BT generation.

4). This list of allowed elements for XML BT construction will be sent to the LLM in the fourth section of the context prompt. Therefore, the model knows what functions it can use to create the BT. This allows for the usage of custom BT tasks, not native to UE, that the user has created. To add a new element, it needs a name that the LLM will send back as a token and a class to link it to the functions it can use (see Fig. 4). These settings produce a list of user-defined tasks and decorators accompanied by the list of possible variables using UE's reflection system.

This full context prompt must be created before the LLM is loaded, ideally as early as possible. Hence, when the prompt window is created, the project settings are loaded first to obtain the list of all BT elements that should be passed to the LLM. Then, the widget blueprint of the window sets the value of the variable "context prompt", which contains the first 3 prompt sections discussed in the previous paragraph. Finally, the model is loaded and given the full context prompt with all 4 prompt sections appended correctly.

3.5 XML Parser

Once the full context prompt is available, the user prompt can be passed to the LLM, which responds directly into the widget editor window. Usually, the LLM provides the XML-formatted BT plus a short description. The output message is thus truncated to retrieve only the XML BT part, which is then given to the XML parser. UE provides a simple library to parse XML files. Because it is a

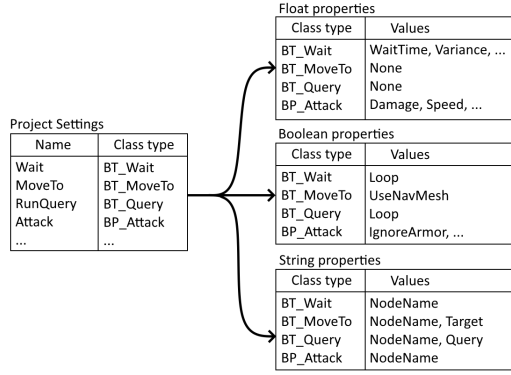


Figure 5: Diagram illustrating how BT node properties are loaded via UE’s reflection system: project settings define the available nodes, whose typed properties (float, bool, string) are extracted and stored in static maps, making them accessible to both the XML parser and the context prompt sent to the LLM.

plugin native to the engine, its code cannot be changed; as such, the following limitations have to be taken into consideration:

- Any error when parsing completely prevents parsing the file.
- The parser only works if there are no empty lines in the file and the XML starts at the first line.
- In a list of XMLs, only the first one is parsed
- For nodes without children, the line must be in the form `<Name parameter="value"/>`. Lines looking like `<Name>parameters="values"</Name>` are not supported.

For this reason, the XML given to the LLM in the context prompt must follow these guidelines, and we must ensure that we trim characters before the beginning of the XML and remove any empty lines before calling the parser. The parser works using recursion; each node parsed lists its children, and if these children are BT nodes, they list their parameters. The name of each parameter is used alongside UE’s reflection system to set the node’s property accordingly. As a list of all properties available both for the XML parser and for the full context prompt is needed, when loading the list of our nodes, the UE’s reflection system is used to access all properties of the types *bool*, *float*, *int* and *string*, which are stored in static maps using the class types as keys, and an array of property names as values (see Fig. 5). It is important to note that nodes referencing blackboard values, even if they correspond to the property types we search for, are not listed in the properties. That is because the blackboard system is separate from the reflection system, and we cannot access these values from code easily.

3.6 LLM Selection

As mentioned earlier, we decided to focus the implementation on local LLM, letting API key-based development for later. The user can download and select the preferred LLM in *.GGUF format. Many

of them are available on Hugging Face⁹, and restrictions regarding any license are up to the user.

We tested many small LLMs and sorted them with respect to their size and to their coherence in the BT generation. Loading an LLM locally can use a lot of resources. Therefore, the maximum LLM size has to be chosen such that it can be handled without hindering the UE editor’s performance. For these reasons, quantized models are preferred. We found that LLMs under the threshold of 2.5 GB in disk size are not impacting performance heavily on a regular laptop while UE remains open. Moreover, the model’s capability of giving coherent answers and generating a mock-up of an XML BT is important. Thus, instruct models are preferred. Among the different tested LLMs, the **Llama-3.2-3B-Instruct-IQ3-M** and the **Schematron-3B-IQ4-NL** models were the most suited ones.

4 Experiments and Results

To test our tool, besides traditional patrol and chase-generated BTs, a gameplay scenario involving an NPC has been chosen. It consists of a boss fight against a dragon. First, several specific decorators and tasks that can be useful in the dragon’s BT were coded. Then, a wide and complex BT was manually created to obtain a reference BT; this hand-crafted reference BT originates from a library of student projects developed at ISART Digital. We evaluate first the ability of our tool to produce small parts of this wide hand-made reference BT using a well-structured user prompt to confirm that, with good guidance, the LLM can produce what the user wants. The results are most of the time satisfying in the first attempt, and when this is not the case, a second or third iteration, indicating what was incorrect, often results in the desired BT.

The Figure 6a shows an example of a user prompt together with the full LLM direct answer (1 iteration), including the XML BT. The corresponding BT asset generated after parsing the XML is shown in Figure 6b. The global structure of the XML BT is correctly obtained, starting and ending with BehaviorTree and Root XML tags and having several Sequences in the inner part. This particular example was chosen deliberately to also discuss the flexibility of our parser.

Sometimes the LLM produces XML BT with wrong tags but correct tag name values. Here, for instance (see Fig.6a), instead of naming the tasks with the correct words, the LLM used “Decorator1”, “Decorator2”, etc. The accompanying tag name, which in principle could be anything (it is just a name for visual reference), contains the actual tasks with correct labels. In this case, our parser detects this and interprets the line correctly as the task the LLM wanted to include there. Indeed, we see in the BT asset (Fig. 6b) that there are actually no decorators; all of them have been translated into tasks with the existing function listed by the user in the settings. Given that the use of decorators is a unique feature of UE, the LLMs’ training data may not reflect knowledge over this control structure.

We also tested the possibility of building the BT gradually by asking the LLM to add small parts, such as introducing a new sequence in between two existing ones. This works relatively well.

⁹<https://huggingface.co> – Hugging Face is an open platform for sharing and discovering machine learning models, datasets, and applications. It hosts a large collection of pre-trained LLMs in various formats, including the GGUF format used by llama.cpp.

USER PROMPT :
 Make a behavior tree for an epic boss fight between the player and the dragon.
 To move the dragon to a target please do follow this sequence order :
 a. Use either Walk, Run, FlyStationary, or FlyNormal
 b. Wait 1 sec
 c. Use ChooseRandomTarget or SetPlayerAsTarget
 d. Finally use MoveToTarget

LLM ANSWER :
 Here is a behavior tree for the epic boss fight between the player and the dragon:

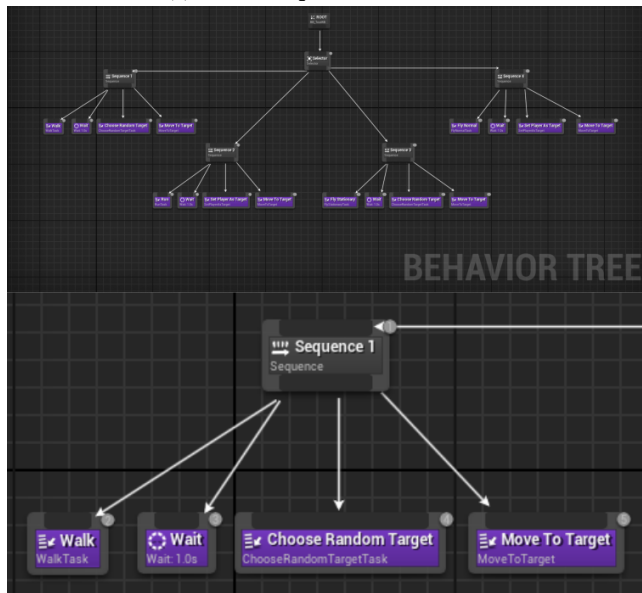
```

<<BehaviorTree>
  <Root name="EpicBossFight">
    <Sequence name="Sequence 1">
      <Decorator1 name="Run"/>
      <Wait name="Wait 1 sec" IgnoreRestartSelf="true" WaitTime="1.0"/>
      <Decorator2 name="ChooseRandomTarget"/>
      <Decorator3 name="MoveToTarget"/>
    </Sequence>
    <Sequence name="Sequence 2">
      <Decorator1 name="Run"/>
      <Wait name="Wait 1 sec" IgnoreRestartSelf="true" WaitTime="1.0"/>
      <Decorator2 name="SetPlayerAsTarget"/>
      <Decorator3 name="MoveToTarget"/>
    </Sequence>
    <Sequence name="Sequence 3">
      <Decorator1 name="FlyStationary"/>
      <Wait name="Wait 1 sec" IgnoreRestartSelf="true" WaitTime="1.0"/>
      <Decorator2 name="ChooseRandomTarget"/>
      <Decorator3 name="MoveToTarget"/>
    </Sequence>
    <Sequence name="Sequence 4">
      <Decorator1 name="FlyNormal"/>
      <Wait name="Wait 1 sec" IgnoreRestartSelf="true" WaitTime="1.0"/>
      <Decorator2 name="SetPlayerAsTarget"/>
      <Decorator3 name="MoveToTarget"/>
    </Sequence>
  </Root>
</BehaviorTree>
  
```

Note:

- * Each 'Wait' node has 'IgnoreRestartSelf' set to 'true' to prevent the dragon from waiting for itself to move.
- * Each 'MoveToTarget' decorator requires a 'ChooseRandomTarget' or 'SetPlayerAsTarget' decorator to specify the target of the movement.
- * The 'Wait' nodes are not actually waiting, they are just a placeholder to introduce a short delay between actions, the actual movement and targeting will happen after the decorator action.

(a) User Prompt and LLM Answer



(b) Generated BT after XML parser

Figure 6: Exemplary Query and Result

Table 1: Results of the user tests on two exercises.

Ex 1	Ex 1 (LLM)	# iter.	Ex 2	Ex 2 (LLM)	# iter.
Fail	Fail	-	Success	Fail	-
Fail	Fail	-	Fail	Success	3
Fail	Fail	-	Success	Fail	-
Fail	Fail	-	Fail	Fail	-
Fail	Success	3	Fail	Success	4
Fail	Fail	-	Success	Success	1
Success	Fail	-	Fail	Fail	-
Fail	Fail	-	Fail	Fail	-
Fail	Fail	-	Fail	Fail	-
Success	Fail	-	Success	Fail	-
Success	Success	3	Success	Success	1

Still, we recommend saving the new BT in a new BT asset each time to avoid overwriting previous ones, as the LLM might also sometimes remove existing parts or modify them a little bit (we did not encounter such behavior so far, but caution is preferable).

We have also evaluated the ability to produce creative BTs when giving only a general context of the gameplay with almost no details. Unfortunately, with a basic prompt, the small LLM was not able to create a BT as wide and complex as the hand-made reference one using all the different available tasks and decorators.

Finally, a small user test has been conducted with 10 undergrad students with 3 to 5 years of experience in game programming. The test was divided into 2 exercises, each being done without and with the LLM-BT plugin, plus one extra exercise to explore the tool with fewer directives. They were asked to create a BT corresponding to a textual description. Here is the first exercise (the second exercise is a variation of the same kind) :

- Sequence 1 (Check dragon health):
 - Check dragon's health.
 - Kill the dragon if health is too weak.
- Sequence 2 (Attack when player is close)
 - Check if the player is in a range of 7.5m around the dragon.
 - Check if player's health is at least 0.1.
 - Select Idle animation
 - Choose a random attack
- Sequence 3 (Chase Player):
 - Check if the player is in a range of 15m around the dragon.
 - Select Walk animation
 - Set player as target
 - Move to the target
- Sequence 4 (Idle when player lost):
 - Check if player is out of range (>15m around the dragon).
 - Set Idle animation

During the survey, time was limited to 5 min for each exercise. If the time exceeded, the exercise was set as failed. In case of a successful exercise with the plugin, the number of consecutive prompts needed to get the desired BT was asked. Finally, users had to say how easy to use and how suited for BT development the plugin was. Table 1 summarizes the results.

The user test seems to indicate that the plugin is not giving immediate success when used for the first time, indicating that

having a bit of experience with the plugin is necessary and that the use of correct keywords in the prompt is important to obtain functional BT. Indeed, during the user analysis, the use of a non-English language or the lack of keywords has been seen as a factor of failure. When asked to rank the usefulness of the tool, 66% say it can be useful, but the tool is currently limited.

5 Conclusion & Future perspectives

A first version of our Unreal Engine LLM-BT editor plugin dedicated to the creation of BT assets assisted by an LLM has been successfully implemented. This confirms the feasibility of our designed workflow for integrating local LLM in UE in editor mode.

The tests have proven that our context prompt, together with our flexible XML parser, allows us to obtain satisfying BT assets even when the XML BT produced by small LLMs was not perfect. Adding prompt components allowed us to ensure a better reliability of the plugin for the user.

Nevertheless, several improvements are possible for a future version of the plugin.

First, the number of elements one can specify in the settings is currently limited by the fact that the context prompt should not be too long to be accepted by the small and local LLM. However, this could be tackled by providing the list of allowed elements (4th section of the context prompt) separately as a hidden pre-user prompt that could possibly be included at every subsequent user prompt to recall the LLM and make sure it does not forget it with an increasing number of iterations.

Second, the XML parser's flexibility could be improved by fuzzily matching XML lines whose tag or tag name values are very close, even if not exact, to an existing label in the list. For that, the LLM could be asked to evaluate the closeness between two labels, or fix errors of existing XML BTs to implement the change directly (in the current implementation, such misspelled lines are discarded).

Third, it could be interesting to use the LLM for generating new tasks and decorators (C++ code) that have not been defined by the user but that the LLM has identified as interesting to have. This, however, requires overall knowledge of the project and has proven to be infeasible with the context window of the small-scale LLMs used in this study.

Finally, to overcome many of the current limitations, fine-tuning of the used LLMs should be performed using a data set of annotated XML BTs. This will likely improve the LLM's capacity to understand when to use decorators or tasks, and how to build and organize large and complex BTs. As an alternative, we also tried to give the exact same context prompt and user prompt to a cloud-based LLM (such as Mistral or ChatGPT). The produced XML BTs are quite good and, as expected, suffer less from hallucinations. In contrast to BTs generated in our case study, they excel in creating larger XML BTs but are still limited in creativity. This may be a result of the limited availability of BT examples (available on the internet). Large-scale LLMs may therefore be used to create a training data set for fine-tuning smaller LLMs.

Last but not least, we plan to publish the plugin on the Unreal FAB platform. We would also like to develop a similar solution for Unity and Godot in future works.

References

- [1] Mael Ahmad Addoum, Jannah Mekhaemar, Maxime Rouffet, and Éric Jacopin. 2021. Khalidun: GOAP for both Procedural Level generation and NPC Behaviors. In *2021 IEEE Conference on Games (CoG)*. IEEE, Copenhagen, Denmark, 1–2. doi:10.1109/CoG52621.2021.9619062
- [2] Abidemi Emmanuel Adeniyi, Biswajit Brahma, Marion Olubunmi Adebisi, Joseph Bamidele Awotunde, Rasheed Gbenga Jimoh, Enoch Olasinde, and Anjan Bandyopadhyay. 2024. Development of Two Dimension (2D) Game Engine with Finite State Machine (FSM) Based Artificial Intelligence (AI) Subsystem. *Procedia Computer Science* 235 (2024), 2996–3006.
- [3] Maren Awiszus, Frederik Schubert, and Bodo Rosenhahn. 2021. World-GAN: a Generative Model for Minecraft Worlds. In *2021 IEEE Conference on Games (CoG)*. IEEE, Copenhagen, Denmark, 1–8. doi:10.1109/CoG52621.2021.9619133
- [4] Maren Awiszus, Frederik Schubert, and Bodo Rosenhahn. 2022. Wor(l)d-gan: toward natural-language-based pcg in minecraft. *IEEE Transactions on Games* 15, 2 (2022), 182–192.
- [5] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. 2024. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*. JMLR.org, Vienna, Austria, 21 pages.
- [6] John N. Carbone, James Crowder, and Ryan A. Carbone. 2020. Radically Simplifying Game Engines: AI Emotions & Game Self-Evolution. In *2020 International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE, Las Vegas, USA, 464–472. doi:10.1109/CSCI51800.2020.00085
- [7] Muyun Dai, Chun Yuan, and Xiaomei Nie. 2024. Managing the personality of NPCs with your interactions: A game design system based on large language models. In *International Conference on Human-Computer Interaction*. Springer, Washington DC, USA, 247–259.
- [8] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. 2024. Large language models and games: A survey and roadmap. *IEEE Transactions on Games* 18 (2024), 18 pages.
- [9] Damian Isla. 2005. Handling complexity in the Halo 2 AI. <https://www.gdcvault.com/play/1020270/Managing-Complexity-in-the-Halo>
- [10] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci. 2024. BTGenBot: Behavior Tree Generation for Robotic Tasks with Lightweight LLMs. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, UAE, 9684–9690. doi:10.1109/IROS58592.2024.10802304
- [11] Devang Jagdale. 2021. Finite state machine in game development. *International Journal of Advanced Research in Science, Communication and Technology* 10, 1 (2021), 7 pages.
- [12] Anssi Kanervisto, Dave Bignell, Linda Yilin Wen, Martin Grayson, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Tabish Rashid, Tim Pearce, Yuhao Cao, et al. 2025. World and human action models towards gameplay ideation. *Nature* 638, 8051 (2025), 656–663.
- [13] Kai Kang, Kai Cheng, Tianhao Shao, Hongjun Zhang, and Ke Zhang. 2024. Planning, Monitoring and Replanning Techniques for Handling Abnormality in HTN-Based Planning and Execution. *Journal of Systems Engineering and Electronics* 35, 5 (2024), 1264–1275. doi:10.23919/JSEE.2024.000090
- [14] Seung Wook Kim, Yuhao Zhou, Jonah Philion, Antonio Torralba, and Sanja Fidler. 2020. Learning to simulate dynamic environments with gamegan. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Seattle, WA, USA, 1231–1240.
- [15] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi. 2023. AudioGen: Textually Guided Audio Generation. arXiv:2209.15352 [cs.SD] <https://arxiv.org/abs/2209.15352>
- [16] Vikram Kumar, Jonathan Rowe, Bradford Mott, and James Lester. 2023. Scenecraft: automating interactive narrative scene generation in digital games with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 19. AAAI Press, Washington DC, USA, 86–96.
- [17] Volodymyr Levytskyi, Mykola Tsiutsiura, Andrii Yerukaiev, Nadiia Rusan, and Tao Li. 2023. The Working Principle of Artificial Intelligence in Video Games. In *2023 IEEE International Conference on Smart Information Systems and Technologies (SIST)*. IEEE, Astana, Kazakhstan, 246–250.
- [18] Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. 2025. CAD-Llama: leveraging large language models for computer-aided design parametric 3D model generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. IEEE, USA, 18563–18573.
- [19] Xinhang Liu, Chi-Keung Tang, and Yu-Wing Tai. 2025. WorldCraft: Photo-Realistic 3D World Creation and Customization via LLM Agents. arXiv:2502.15601 [cs.CV] <https://arxiv.org/abs/2502.15601>
- [20] Ibrahim M. Mahmoud, Lianchao Li, Dieter Wloka, and Mostafa Z. Ali. 2014. Believable NPCs in serious games: HTN planning approach based on visual perception. In *2014 IEEE Conference on Computational Intelligence and Games*. IEEE, Milano, Italy, 1–8. doi:10.1109/CIIG.2014.6932891

- [21] Adapala Rishi Manikanta, Baba Ameer Shaik, Yaswanth Kancharla, Revanthasai Gurram, and Gayathri Ramasamy. 2025. AI-Driven Game Mechanics: Implementing Finite State Machines in Pac-Man. In *2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, Vol. 3. IEEE, Gwalior, Indien, 1–6. doi:10.1109/IATMSI64286.2025.10985319
- [22] Ryan Marcotte and Howard J Hamilton. 2017. Behavior trees for modelling artificial intelligence in games: A tutorial. *The Computer Games Journal* 6 (2017), 171–184.
- [23] Thomas Marrinan, Pakeeza Akram, Oli Gurmessa, and Anthony Shishkin. 2024. Leveraging AI to Generate Audio for User-generated Content in Video Games. arXiv:2404.17018 [cs.HC] <https://arxiv.org/abs/2404.17018>
- [24] Xenija Neufeld, Sanaz Mostaghim, Dario L Sancho-Pradel, and Sandy Brand. 2017. Building a planner: A survey of planning systems used in commercial video games. *IEEE Transactions on Games* 11, 2 (2017), 91–108.
- [25] Miguel Nicolau, Diego Perez-Liebana, Michael O'Neill, and Anthony Brabazon. 2016. Evolutionary behavior tree approaches for navigating platform games. *IEEE Transactions on Computational Intelligence and AI in Games* 9, 3 (2016), 227–238.
- [26] D Ogunlesi and X Wang. 2024. GPT-NPC: Enhancing NPC Human-Likeness and Autonomy in Video Games. In *Proceedings of the Workshop on "eXtended Reality & Intelligent Agents"*. XRIA24, Santiago de Compostela, Spain, 10 pages.
- [27] L. O'Brien. June 2024. How Ubisoft's New Generative AI Prototype Changes the Narrative for NPCs. <https://news.ubisoft.com/en-us/article/5qXdxshjJBXoanFZApdG3L/how-ubisofts-new-generative-ai-prototype-changes-the-narrative-for-npcs>.
- [28] Ciprian Paduraru and Miruna Paduraru. 2019. Automatic difficulty management and testing in games using a framework based on behavior trees and genetic algorithms. In *2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS)*. IEEE, Guangzhou, China, 170–179.
- [29] Nathan Partlan, Luis Soto, Jim Howe, Sarthak Shrivastava, Magy Seif El-Nasr, and Stacy Marsella. 2022. EvolvingBehavior: towards co-creative evolution of behavior trees for game NPCs. In *Proceedings of the 17th International Conference on the Foundations of Digital Games*. ACM, Athens, Greece, 1–13.
- [30] Rasmus Ploug, Emil Rimer, Anthon Kristian Skov Petersen, and Marco Scirea. 2025. Open-Ended NPC Dialogue Favors Casual Players: A Pilot Comparison of Three LLM-Driven Dialogue Systems. In *2025 IEEE Conference on Games (CoG)*. IEEE, Lisbon, Portugal, 1–4.
- [31] Rafael Ribeiro, Alexandre Valle de Carvalho, and Nelson Bilber Rodrigues. 2024. Image-based video game asset generation and evaluation using deep learning: a systematic review of methods and applications. *IEEE Transactions on Games* 17 (2024), 622–630.
- [32] Emil Rimer, Rasmus Ploug, Anthon Kristian Skov Petersen, and Marco Scirea. 2025. Talking to NPCs: Three LLM-Driven Approaches to Dynamic RPG Dialogue. In *2025 IEEE Conference on Games (CoG)*. IEEE, Lisbon, Portugal, 1–2.
- [33] Yoonas A Sekhavat. 2017. Behavior trees for computer games. *International Journal on Artificial Intelligence Tools* 26, 02 (2017), 1730001.
- [34] Yifei Shen, Jiawei Shao, Xinjie Zhang, Zehong Lin, Hao Pan, Dongsheng Li, Jun Zhang, and Khaled B Letaief. 2024. Large language models empowered autonomous edge AI for connected intelligence. *IEEE Communications Magazine* 62, 10 (2024), 140–146.
- [35] Jorde Stojanov, Lazaros Rafail Kouzelis, and Ourania Spantidi. 2024. Toward Dynamic NPC Interactions: Integrating GPT-Driven Agents in 3D Virtual Environments. In *International Symposium on Visual Computing*. Springer, Cham, Switzerland, 199–211.
- [36] BungieBungie Studios. 2004. Halo 2. video game, Xbox, Microsoft.
- [37] Dimas Adi Suyikno and Abas Setiawan. 2019. Feasible NPC Hiding Behaviour using Goal Oriented Action Planning in case of Hide-and-Seek 3D Game Simulation. In *2019 Fourth International Conference on Informatics and Computing (ICIC)*. IEEE, Semarang, Indonesia, 1–6. doi:10.1109/ICIC47613.2019.8985962
- [38] MF Syahputra, A Arippa, RF Rahmat, and Ulfi Andayani. 2019. Historical theme game using finite state machine for actor behaviour. In *Journal of Physics: Conference Series*, Vol. 1235. IOP Publishing, Bristol, UK, 012122.
- [39] Graham Todd, Sam Earle, Muhammad Umair Nasir, Michael Cerny Green, and Julian Togelius. 2023. Level generation through large language models. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*. Association for Computing Machinery, New York, NY, USA, 1–8.
- [40] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. 2025. Diffusion Models Are Real-Time Game Engines. arXiv:2408.14837 [cs.LG] <https://arxiv.org/abs/2408.14837>
- [41] Kaijie Xu and Clark Verbrugge. 2025. Constraint Is All You Need: Optimization-Based 3D Level Generation with LLMs. In *Proceedings of the 20th International Conference on the Foundations of Digital Games*. Association for Computing Machinery, New York, NY, USA, 1–13.